Parameter substitution is enough for iPRA

A raw clause calculus and a complete proof checker

Research note on the ConciseIPRA proposal

November 12, 2026

Abstract

We give a raw presentation of intuitionistic primitive recursive arithmetic whose primitive equality operation substitutes inside terms only at free numerical variables. Equalities with compound sources are used only by root composition or symmetry. A fresh-alias construction derives arbitrary term-template transport, giving exact consequence equivalence with the repository's native calculus and, through its established bridge, whole-domain bi-interpretability with full first-order iPRA. The complete checker uses five instruction families, including the two explicit iterator computation equations, and has no primitive reflexivity rule. In the strongest matched comparison it is 2857 rather than 2907 normalized bytes, with slightly larger token and AST counts. The paper states every rule, proves both interpretation round trips, supplies checked proof transformations, and distinguishes this local simplification from an unproved global minimum.

Deliverable status: written mathematical proof; executable checker and proof translations; no newly claimed Lean or Rocq verification.

Contents

1 Result, comparison target, and limits **3**

2 A raw calculus with parameter substitution **4**

2.1 Syntax, clauses, and ordinary readings 4

2.2 A fixed, restricted equality operation 5

2.3 Every primitive instruction 5

3 The equality bootstrap and the fresh-alias theorem **6**

4 Recovering the native logical rules **7**

5 Exact consequence equivalence and iPRA **8**

5.1 Structural substitution and effective checking 10

6 The complete checker and executable proof transformations **10**

6.1 An exact public contract 10

6.2 One opening operation, with an explicit invariant 11

6.3 Measured entire modules 11

6.4 Proof compilers and validation 12

6.5 Reproduction 13

7 Further research questions **13**

8 Provenance and exact status **14**

A The complete trusted checker **15**

1 Result, comparison target, and limits

The result. General numerical equality transport can be derived from a rule that substitutes only for a *free numerical variable* inside a term, together with equality composition at the root. The argument gives a fresh local name to the term to be transported. The logical calculus itself supplies that name and later discharges it. Consequently the trusted equality instruction receives two proofs and no term template. It does not search for occurrences of a compound source term.

The resulting theory retains raw iterator syntax and ordinary capture-avoiding substitution. Its only public formula constructors are equality and the paired binder

$$D_{x,y}B := \exists x(B(x,0) \to \forall y\, B(x,y)).$$

There are five serialized instruction families: Compute, Param, Use, Open, and Close. Compute contains the two usual iterator equations; there is no primitive reflexivity instruction. The detailed statement below is the complete axiom-and-rule specification, including every side condition and each branch of the finite operations.

The equivalence established. The paper proves exact preservation and reflection of ordinary sequents, and both formula round trips, between this presentation and the native calculus in the supplied repository. Composing this comparison with its formalized arithmetic bridge establishes bi-interpretability with full first-order intuitionistic PRA. Both numerical domains are whole, equality is actual equality, and the numerical composite isomorphisms have graph $x = y$. This is stronger than agreement on true numerical equations in the standard natural numbers.

Pinned baseline. The source snapshot is ProveIt commit `5d85023b0688e03b75f0ec3a04bb43ef0bb51705`, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The review included its index of 56 filed packages and the five archives then present under `docs/incoming`. The relevant predecessors are the raw clause presentation, template-free equality, one-term transport, and the incoming root-free and uniform-step presentations [1, 3, 2, 5].

The clause interface and fresh-alias argument are inherited ingredients. The specific reduction isolated here is that the fresh-alias argument requires proper-subterm replacement only at variables: replacement of compound patterns can be removed as well. The written proof and the executable proof compiler make that restriction explicit. No claim of priority outside the inspected repository is made.

What "simpler" means here. The equality interface is mathematically weaker and has no supplied template. In a matched implementation with unused machinery removed from the control, the complete program decreases from 2907 to 2857 normalized bytes. It uses four more significant tokens and eleven more AST nodes. Thus the result is a structural reduction and a modest byte improvement, not a simultaneous improvement in every metric. The original filed raw clause checker is 3046 bytes; comparisons with it also include implementation changes and must not all be credited to the equality theorem.

There is no established absolute minimum. The repository's compact codecs remain much smaller. Moreover, an implementation refinement of the existing two-case Eval calculus measures 2731 bytes in this same investigation. The new presentation is therefore recommended for its restricted equality rule and ordinary substitution, not advertised as the smallest known complete Python program.

Proof status. All new mathematical implications used in the construction are written out below. The complete Python checker and proof transformations have been executed. The new kernel has not been formalized in Lean or Rocq. The inherited native/iPRA equivalence is a separately formalized dependency, whose relevant source modules and theorem names are included.

2 A raw calculus with parameter substitution

The principal simplification is a restriction on primitive equality reasoning. Proper-subterm substitution need only replace a free numerical parameter. Nonvariable sources are needed only for common-root equality composition. Arbitrary term-template transport is then derived by giving its argument a fresh local name. Unlike a presentation by computation normal forms, this calculus retains ordinary raw syntax and ordinary capture-avoiding substitution throughout.

The fresh-alias method is an existing alternative argument in [2], and clauses occur in [3]. The new restriction is that arbitrary-pattern replacement can itself be removed: the alias argument needs only variable substitution and root equality composition. The following rule basis, exact translations, and complete checker comparison make that restriction concrete.

2.1 Syntax, clauses, and ordinary readings

Numerical terms are finite raw expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where only the displayed u in h is bound. Formulas are

$$F ::= (s = t) \mid D_{x,y}F.$$

Both displayed coordinates of D are bound. All substitutions below are ordinary capture-avoiding substitutions. In particular, $I(0, a; u.h)$ and a are distinct terms, including after substitution. There is no reduction or normalization while constructing a term or substituting into a formula.

A reusable context is a finite list of clause occurrences

$$[A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

The ordinary interpretations are

$$(s = t)^\circ = (s = t), \qquad (D_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big),$$

$$[A; y.B]^\circ = A^\circ \to \forall y\, B^\circ(y).$$

Equality has its actual numerical meaning. Clauses are context objects, not a third formula constructor. Let $H = (0 = 0)$ and abbreviate an ordinary assumption by the constant clause

$$\mathrm{ord}(F) = [H; v.F], \qquad v \notin \mathrm{FV}(F).$$

The notation Γ, F means $\Gamma, \mathrm{ord}(F)$. Occurrences are retained even if their values coincide. The designated contradiction is $0 = S0$.

2.2 A fixed, restricted equality operation

For raw terms s, t, u, define

$$\psi(s,t,u) = \begin{cases} u[t/s], & s \text{ is a free numerical parameter,} \\ t, & s \text{ is not a parameter and } u \equiv s, \\ u, & s \text{ is not a parameter and } u \not\equiv s. \end{cases}$$

The first case substitutes for a variable, with capture avoided, and does not rescan an inserted target. It is defined even if t contains s. For every source, $\psi(s,t,s) = t$. For a nonvariable source, no proper subterm is inspected; the last case merely reverses the minor equation in the rule below. Bound variables are not free numerical parameters. The operation has no supplied equality template and no arbitrary-pattern replacement pass.

Define also the partial one-step computation operation on an iterator-headed term b:

$$\kappa(I(0,a;u.h)) = a, \qquad \kappa(I(Sn,a;u.h)) = h[I(n,a;u.h)/u].$$

It is undefined for all other term shapes. The recursive iterator in the second result is left as raw syntax; κ does not repeatedly evaluate its result.

2.3 Every primitive instruction

The entire calculus $\mathcal{P}$ has the following five instruction families:

$$\frac{}{\Gamma \vdash b = \kappa(b)} \quad \text{when } \kappa(b) \text{ is defined.} \tag{Compute}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash u = r}{\Gamma \vdash r = \psi(s,t,u)}. \tag{Param}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } [A; y.B(y)] \text{ is a selected occurrence in } \Gamma. \tag{Use}$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, [B(b,0); y.B(b,y)] \vdash F}{\Gamma \vdash F}. \tag{Open}$$

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Close}$$

In Open, b is fresh for the retained context, conclusion, and all passive parameters of B. In Close, z is fresh for the retained context, witness, and all passive matrix parameters. All terms, formulas, and clauses must be well scoped. The matrix-root and literal-zero tests are tests on raw syntax. These are part of the specification checked by the program.

There is no separate reflexivity axiom, hypothesis rule, equality template, first-class Q constructor, numerical case rule, or induction schema. Compute explicitly contains *two* computation axiom schemas. Calling it one instruction family is an implementation grouping, not an assertion that two mathematical equations became one equation. Likewise the finite cases of Param and Close are charged to the complete checker.

3 The equality bootstrap and the fresh-alias theorem

Lemma 3.1 (Root equality algebra)**.** *Common-left composition, reflexivity, symmetry, and transitivity are admissible under any retained context:*

$$s = t,\ s = r \vdash r = t, \qquad \vdash a = a, \qquad s = t \vdash t = s, \qquad s = t,\ t = r \vdash s = r.$$

Proof. The first rule is a root instance of Param, since $\psi(s, t, s) = t$. For reflexivity put $J(a) = I(0, a; u.0)$. Compute proves $J(a) = a$; using that equation as both premises of common-left composition gives $a = a$. For symmetry, use major $s = s$ and minor $s = t$. For transitivity, reverse $s = t$ and use major $t = r$ and minor $t = s$. Every step uses only Compute and root equality composition. □

Lemma 3.2 (Elementary proof operations)**.** *Ordinary assumption use, weakening, injective fresh renaming, and cut for ordinary formula assumptions are admissible.*

Proof. Derived reflexivity proves H, so Use on $[H; v.F]$ proves F. Weakening and injective fresh renaming follow by induction on a finite derivation. Such renaming preserves free-parameter status, exact root matching, equation roots, and the literal zero. For cut, distinguish the occurrence $[H; v.F]$ being discharged. Replace every Use selecting that occurrence by the supplied proof of F, weakened into the local context. At other instructions recurse through the premises, first freshening eigenvariables away from the inserted proof. This establishes cut without presupposing arbitrary numerical substitution into derivations. □

Lemma 3.3 (Fresh aliases)**.** *For a fixed raw term a and b fresh for Γ, F, and a,*

$$\frac{\Gamma, b = a \vdash F}{\Gamma \vdash F} \qquad \text{(Alias)}$$

is admissible. The local assumption is an ordinary formula assumption. Only the two computation equations, root equality composition, the logical instructions, and ordinary cut are needed to construct it.

Proof. Put $C(y, a, x) = I(y, a; u.x)$, with u fresh, and let

$$B(x, y) := (C(y, a, x) = a).$$

Close with witness a proves $D_{x,y}B$. Its successor branch $C(Sz, a, a) = a$ is exactly the successor Compute instance with constant update a. The local assumption is unused, so this works whichever mode Close selects, including when $a \equiv 0$.

There is also a version using case-mode Close alone. Take the raw nonliteral witness $J(a) = I(0, a; v.0)$ instead. Successor Compute gives $C(Sz, a, J(a)) = J(a)$, zero Compute gives $J(a) = a$, and root transitivity joins them to the required branch. This again ignores its local assumption and always selects cases. Thus the alias lemma does not require equation progress, even though the shorter certificate can use the direct witness a.

Open at fresh b. Its retained clause is

$$[\, C(0, a, b) = a;\ y.C(y, a, b) = a \,].$$

Zero Compute proves the antecedent. Use at target $1 = S0$ gives $C(1, a, b) = a$. Successor Compute gives $C(1, a, b) = b$. Common-left composition, with the former as major and the latter as minor, gives $b = a$. Cut that ordinary assumption into the supplied branch, after weakening it into the clause context. Open discharges b and returns F in the original context. □

This is the existing fresh-alias idea in a form adapted to the present restricted equality interface. Its significant feature for the bootstrap is that it needs no proper-subterm substitution, term-template transport, or equality induction. It therefore cannot conceal the equality principle that it is subsequently used to derive.

Theorem 3.4 (Parameter substitution suffices for all templates)**.** *For every fixed finite numerical term template $h(v)$,*

$$\Gamma \vdash s = t \quad \Longrightarrow \quad \Gamma \vdash h(s) = h(t).$$

Repeated, absent, and binder-nested occurrences of the displayed variable are all permitted.

Proof. Work temporarily under the ordinary assumption $b = s$, with b fresh for the context, h, s, and t. Transitivity gives $b = t$. Derived reflexivity gives $h(b) = h(b)$. Param with major $b = s$ and this minor gives

$$h(b) = h(s),$$

because substitution for the free variable b marks exactly the distinguished template occurrences. The same construction with major $b = t$ gives $h(b) = h(t)$. Common-left composition joins these equations to $h(s) = h(t)$. Its endpoints contain no b, so Lemma 3.3 discharges the temporary alias.

Only the parameter case of Param is used for proper-subterm substitution. The argument is not a pass replacing every occurrence of the possibly compound term s in $h(s)$: that would also replace passive occurrences of s already present in the template. The untrusted proof compiler builds $h(b)$, while each checker instruction receives only two proofs. □

Corollary 3.5 (The complete equality interface)**.** *Crossed one-term transport and transport in an arbitrary equation template are admissible:*

$$s = t,\ h(s) = r \vdash r = h(t), \qquad s = t,\ a(s) = b(s) \vdash a(t) = b(t).$$

Proof. Join the congruence equation $h(s) = h(t)$ with the supplied minor to obtain the crossed conclusion. Apply crossed transport first to a and then to b to obtain the second rule. □

In particular, the unrestricted one-pass replacement rule of [2] is derived. A root-first pass selects finitely many pairwise disjoint source occurrences in a term u. Replacing precisely those occurrences by a fresh hole gives a finite template h with $h(s) = u$ and $h(t)$ equal to the pass's output. The derived crossed rule therefore gives the same conclusion. This template extraction can occur in an untrusted proof compiler; it is absent from the checker.

4 Recovering the native logical rules

Lemma 4.1 (Every case introduction)**.** *For every matrix B and witness t,*

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}$$

is admissible with the usual freshness of z.

Proof. Only a literal equation E at witness 0 is not already Close's case mode. Work under a fresh alias $b = 0$. From the local $E(b, 0)$, equation-template transport gives $E(0, 0)$. Cut in the supplied branch, obtaining $E(0, Sz)$. Reverse the alias and transport back to $E(b, Sz)$. Close at the fresh variable b is in case mode and gives the required $D_{x,y}E(x, y)$. Alias discharges b. This uses only the already established atomic equality interface; transport through a D formula is not assumed. □

For fresh unused w, define the formula abbreviation

$$q_y B(y) := D_{w,y} B(y).$$

Its three interfaces are derivable:

1. From $\Gamma, B(0) \vdash B(Sz)$ derive $\Gamma \vdash q_y B$. Close at unused witness 1 is always in case mode.
2. From $q_y B$ and $B(0)$ derive $B(t)$. Open the dummy witness, apply its clause by Use, and close Open.
3. From a retained clause $[B(0); y.B(y)]$ derive $q_y B$. Under the first construction's local $B(0)$, use that clause at Sz.

For a literal equation $E(y)$, Close at unused witness 0 directly gives $q_y E$ from $\Gamma, E(z) \vdash E(Sz)$. With a supplied base and the second interface, this is contextual equation induction. The first two interfaces similarly give numerical cases for any formula.

Transport in every Eq/D formula follows by external formula induction. The equation case has been proved. For $F(v) = D_{x,y} B(v, x, y)$, open $F(s)$ at b and introduce $F(t)$ at the same witness by cases. Convert its target base $B(t, b, 0)$ backwards using the induction hypothesis and symmetry, use the retained source clause at Sz, and convert the successor instance forwards. This recursion uses only the strictly smaller matrix.

5 Exact consequence equivalence and iPRA

Let $\mathcal{N}$ denote the repository's raw native Eq/Q/D calculus: Hyp, the two iterator computations, one-term crossed transport, closure elimination, paired elimination, case introduction, and direct equation-progress introduction. Its terms are exactly the raw terms above, and its ordinary closure reading is $Q_y B = B(0) \to \forall y B(y)$.

Define a formula translation, leaving numerical terms unchanged, by

$$\tau(s = t) = (s = t), \qquad \tau(D_{x,y} B) = D_{x,y} \tau B, \qquad \tau(Q_y B) = D_{w,y} \tau B,$$

where each newly inserted witness coordinate is fresh and unused. Let j include Eq/D formulas into native formulas. The translation commutes with ordinary capture-free numerical substitution and preserves whether a formula's root is an equation or compound.

Theorem 5.1 (All ordinary sequents are preserved and reflected)**.** *For every finite native formula context and formula,*

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longleftrightarrow \quad \mathrm{ord}(\tau\Gamma) \vdash_{\mathcal{P}} \tau A.$$

For Eq/D formulas, $\tau j F = F$ literally up to bound renaming, and for every native formula A,

$$A \;\dashv\vdash_{\mathcal{N}}\; j\tau A.$$

Proof. Forwards, induct on the native derivation. Hyp becomes a proof of H and Use on the appropriate ordinary clause. Computation becomes the corresponding Compute case. Transport uses Theorem 3.4 and common-left composition. Closure elimination uses the derived q specialization. Case introduction uses Lemma 4.1, and equation progress uses Close at unused witness 0.

For native paired elimination, the translated major is opened into its clause. The third closure interface derives the translation of the old branch's Q assumption from that clause. Weaken the inductively translated branch, cut in that ordinary assumption, and close Open.

Backwards, track each clause occurrence's origin. An initial ordinary clause, or an ordinary clause added in Close, is represented by the corresponding native formula assumption. A clause added in Open is represented by the corresponding native Q assumption. Use on the former is native Hyp, and Use on the latter is native closure elimination. Compute is the matching native axiom. Param is an instance of native crossed transport: use template $u[v/s]$ if s is a free parameter, the identity template if $u \equiv s$ and s is not a parameter, and the constant template u in the remaining case. Here v is chosen fresh, so these substitutions avoid capture.

Open is native paired elimination. A case-mode Close is native case introduction. A progress-mode Close at witness 0 first uses native equation progress to obtain $Q_y E(0, y)$; under $E(0, 0)$ specialize it at Sz and introduce $D_{x,y}E(x, y)$ by native cases. This proves reverse preservation for all Eq/D ordinary sequents.

The composite τj is literal identity. For the other composite, the essential native equivalence is

$$Q_y B(y) \;\dashv\vdash\; D_{w,y} B(y), \qquad w \text{ unused.}$$

Forwards, specialize the closure under its base and introduce the dummy witness by cases. Backwards, paired elimination opens exactly the desired Q assumption. Uniform pointwise interderivability lifts through D by elimination and case introduction, converting the base backwards and the successor instance forwards; it lifts through Q in the same way with a dummy witness. External structural induction proves $A \dashv\vdash j\tau A$. Ordinary cut with these equivalences also gives the reverse implication for the displayed native sequent. □

The same proof represents an arbitrary externally supplied clause by the native ordinary macro $A \to \forall y B(y)$ if a statement for arbitrary clause contexts is desired. The theorem already covers all ordinary first-order consequences, not merely empty-context theorems.

Corollary 5.2 (Equiconsistency and bi-interpretability)**.** *The ordinary realization of $\mathcal{P}$ is equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, equality is actual equality, and both numerical composite isomorphisms have graph $x = y$.*

Proof. Compose the explicit interpretations and both formula round trips above with the pinned repository's native/iPRA bi-interpretation [4]. Numerical terms have not changed in the new comparison, so its numerical maps are literal identities. The inherited bridge first recovers ordinary intuitionistic logic and the scalar arithmetic theory with the iterator equations, coverage

$$\forall x\, (x = 0 \vee \exists y\, x = Sy),$$

separation $0 = S0 \to \bot$, and contextual equation induction. It then constructs predecessor, reflection of equality, quantifier-free induction, pairing, and scalar implementations of every fixed primitive-recursive definition. The resulting formula and term round trips, composed with those above, establish the full first-order bi-interpretation.

The designated contradiction $0 = S0$ is literally unchanged by τ and j, so finite contradiction proofs transfer in both directions. The ordinary realization includes the stated falsity bridge; a theory of positive equations alone would not exclude singleton structures. □

The new equality restriction and certificate compiler have written proofs and executable checks, not a newly claimed Lean or Rocq formalization. The inherited native/iPRA bridge is a separately formalized dependency. No theory is asserted to prove its own consistency.

5.1 Structural substitution and effective checking

Arbitrary numerical substitution is not justified by copying instruction tags: it can turn a Param source from a variable into a compound term, and can turn a nonzero-spelled Close witness into the literal zero. Substitution admissibility for ordinary sequents instead follows from the exact comparison with the established native calculus. The bootstrap above used only weakening, fresh renaming, and ordinary-assumption cut, which were proved directly.

The raw certificate checker uses finite structural traversals, capture-free substitutions, literal comparisons, and one-step computation. It does not call a solver or a uniform evaluator for primitive-recursive programs. Under standard finite-tree encodings its verification relation is primitive recursive. This does not imply a useful polynomial bound: proof trees and substituted terms can still grow substantially. The smaller source comparison must count parsing, scope checks, clause formation and opening, the two Compute cases, all Param cases, and every eigenvariable check. A five-instruction presentation by itself is not a theorem of global minimality.

6 The complete checker and executable proof transformations

6.1 An exact public contract

The standalone program is `code/raw_parameter.py`. It imports only Python's standard library. It accepts one decoded JSON object with exactly the keys `goal` and `proof`, and verifies a closed Eq/D theorem. There are no externally supplied assumptions. Open sequents are available to the mathematical calculus and to the internal proof-compilation interface; proof branches introduce their own scoped parameters and clauses.

Successful checking prints `ACCEPT` and returns exit code zero. Rejection prints `REJECT` and returns a nonzero exit code. No acceptance decision depends on a Python `assert`. The standard JSON parser has its usual last-key semantics for duplicate textual keys; the decoded object, terms, formula, and derivation are checked. Extremely large finite certificates can exceed Python recursion or memory limits.

The concrete numerical representation is:

Mathematical object	JSON representation
Zero	`0`
Free parameter at level $k \geq 1$	integer $-k$
Locally bound index $k \geq 1$	integer k
$S(t)$	`[0,t]`
$I(n, a; u.h)$	`[n,a,h]`
$s = t$	`[s,t]`
$D_{x,y}B$	`[B]`

The expected sort disambiguates equality from successor. In the matrix of D, the inner variable y has bound index 1 and the outer variable x has index 2. Only the third child of an iterator binds its state variable. Booleans and floating-point values are not integers for this grammar.

The proof instructions are:

Certificate	Meaning
`["c",n,a,h]`	Prove the appropriate computation equation for `[n,a,h]`; the counter must literally be zero or a successor.
`["x",p,q]`	Apply Param to the two inferred equations.
`["u",j,t,p]`	Use clause occurrence j at target t; p must prove its antecedent.
`["e",F,p,q]`	Open the paired formula proved by p; q proves F under the fresh parameter and retained clause.
`["i",F,t,q]`	Close at witness t; q proves the specified successor instance under the automatically selected premise.

For example, the following is a complete certificate for $0 = 0$:

```
{"goal":[0,0],
 "proof":["x",["c",0,0,1],["c",0,0,1]]}
```

The two children prove $I(0, 0; u.u) = 0$; their root join proves reflexivity.

6.2 One opening operation, with an explicit invariant

Positive indices in `opening` select a bound coordinate. Under a binder the selected index increases, and indices beyond the removed coordinate decrease after opening. A negative index selects a free parameter; it remains fixed under binders, and no other indices change. This shares the syntax traversal used by clause opening and by Param.

Every inserted term is locally closed with respect to positive bound indices: its free numerical parameters have negative labels. Its own internally bound variables are still allowed. Consequently insertion needs no separate shifting pass. This is an interface invariant, not a general assertion that de Bruijn substitution never needs lifting.

The leading zero of `[0,t]` is a successor marker. Param's proper substitution acts only on negative integers, so it cannot rewrite this marker. Root equality composition is handled separately. The negative test for free parameters is therefore essential to both the specification and this representation.

6.3 Measured entire modules

The measurement uses CPython 3.12.14: remove the module docstring, parse the complete source, apply `ast.unparse`, and append one line feed. Significant tokens exclude comments, whitespace, indentation, and newline tokens. AST counts include every node from `ast.walk`. Parsing, validation, scoping, substitutions, all inference branches, and the command-line interface are counted. Gzip-9 with timestamp zero is informational only.

Complete module	Bytes	Tokens	AST	Gzip
Matched template control, unused free-substitution code removed	2907	765	881	988
New raw parameter calculus	**2857**	**769**	**892**	**998**
Shared-opener template control	2937	777	895	1001
Filed raw three-case Eval	3046	853	993	990
Incoming two-case Eval	2999	848	986	979
Incoming root-free raw checker	2956	837	977	962
Incoming zero-normal fixed-base checker	2948	852	987	967
Incoming uniform Step	2950	829	902	1005

The clean comparison is 50 bytes, or 1.72%, below the pruned matched template control, with the token and AST tradeoff already stated. The 80-byte reduction against the shared-opener control isolates the rule change while keeping that helper unchanged, but that control carries a negative-index facility it does not need. It is not the strongest comparator. The 189-byte difference against the filed raw CE5 checker includes both the new basis and implementation refinements.

The new module is not a repository-wide size record. The incoming `scope_lexical.py` checker is 860 bytes under a different certificate protocol. The historical 516-byte compact guarded codec has a different public-language contract and does not contain the same public-grammar validator. Neither should be compared as if it were an instance of the present transparent proof-tree interface. Even within the investigation, a refined two-case Eval implementation is 2731 bytes, 741 tokens, and 862 AST nodes; its source is supplied as a comparison, without representing it as a new mathematical theory.

6.4 Proof compilers and validation

`translate_raw.py` translates proofs in both directions between the matched template calculus and the new calculus. Forward translation expands a template transport into the fresh-alias construction. Reverse translation replaces each Param instance by its ordinary template instance. It preserves the end formula exactly and checks the input and output proofs with their respective standalone verifiers. The translator is an untrusted producer and is not imported by the trusted checker.

The executed validation suite records:

- 38 positive closed examples and 26 adversarial rejections;
- 76 checked forward/backward proof translations, preserving goals;
- 2402 independent syntactic opening comparisons and 800 independent denotational opening checks, with no skipped semantic cases;
- 12 checks of root versus parameter substitution boundaries;
- 128 public command-line runs, covering normal and optimized Python execution.

The cases include genuinely used induction hypotheses, nested D binders, closure specialization, repeated and passive occurrences of a compound source, sources equal to zero, iterator-update binders, invalid counter shapes, escaped eigenvariables, malformed certificate fields, and a direct attempt to perform the removed proper-zero replacement.

These tests support the implementation claims. They do not prove the universal equivalence theorem; that theorem is supplied by the written proof. A separate mathematical and implementation audit is included in the evidence directory. No new proof-assistant verification is claimed.

6.5 Reproduction

From the extracted archive root:

```
python3 reproduce.py
python3 code/raw_parameter.py < examples/reflexivity.json
python3 code/raw_parameter.py < examples/used_induction.json
python3 reproduce.py --pdf
```

The first command requires only the standard Python library. The last also requires pdfLaTeX and the ordinary packages named in the source. The delivered main `.tex` file is standalone: it embeds the complete mathematical text and checker listing, and does not require the separate paper fragments to compile.

7 Further research questions

1. **Formalize the restricted equality kernel.** Add a new derivability predicate to the pinned Lean development and prove the fresh-alias theorem, unrestricted crossed transport, and exact equivalence with `NPrf`. The useful boundary is the alias lemma: its proof needs computation and root joining, but no variable-substitution instance. This isolates the noncircular bootstrap for a formal audit.
2. **Find the least residual equality interface.** The useful instances of Param are root joining and substitution at a fresh variable. Its fallback is only symmetry. Can a smaller total operation express exactly those indispensable instances without a larger syntax test? A proposal must account for arbitrary ground equality assumptions, rather than showing only that computation equations survive.
3. **Control certificate expansion.** The primitive checker becomes simpler while the proof producer builds an alias scope around each arbitrary template transport. Quantify the shortest expansion and compare tree certificates with explicit sharing. A smaller kernel can be accompanied by larger proofs; both costs deserve measurement.
4. **Obtain a substitution-stable single introduction.** The literal-witness convention remains sensitive to numerical substitution. The current diagonal-guard work supplies a stable alternative, but it costs a more elaborate shape test. A smaller stable guard must also permit recovery of case introduction; a conditional closure with an unproved base cannot simply be used as a progressive closure.
5. **Separate arithmetic reductions from coding reductions.** The predecessor/equality-reflection bootstrap remains the obstacle for weaker successor-rigidity or zero-equation induction proposals. Solving it would change the arithmetic foundation, while the present theorem changes the equality interface. The latter does not settle the former.
6. **State a reproducible minimization problem.** A meaningful lower bound needs a fixed public contract, source language, normalization policy, and cost metric. The present result improves one matched byte coordinate and restricts the equality schema; it supplies no lower bound proving that these six explicitly split schemas are optimal.

8 Provenance and exact status

The construction is a new restriction and integration of ingredients in the supplied project. The definitions of scalar iteration, paired closure, clause contexts, and the single witness-directed introduction are inherited. The idea of introducing a fresh numerical alias also appears in the optional proof in `template_free`. The present argument isolates the smaller equality interface it actually requires.

The archive includes unchanged inherited bridge fragments for reference: the native choice basis, ordinary logical bridge, scalar predecessor, equality reflection, and compilation of primitive recursion. Their role is to document the existing arithmetic dependency. They are not counted as newly proved results. The main paper gives the complete new reduction and names the precise inherited Lean statements used at the last step: `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`.

Original ProveIt material is distributed under its included MIT No Attribution license. The new checker, proof producers, proof notes, and reproduction scripts may be reused under the same terms. The repository was read at the pinned revision; this package does not publish changes to it.

References

[1] Vladimir Reshetnikov and ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. Pinned commit `5d85023b0688e03b75f0ec3a04bb43ef0bb51705`. Project index and rule inventory.

[2] ProveIt, *Template-free equality*, especially the optional fresh-variable alias proof in `Research/template_free/evidence/global_rewrite_marker_proof.md`. Pinned proof note. An unchanged copy is included in the archive.

[3] ProveIt, *Five rules for iPRA with ordinary substitution*, `Research/raw_clausal`. Pinned raw clause presentation. Its complete checker supplies the 3046-byte comparison and the inherited positive example corpus.

[4] ProveIt Lean formalization, `Lean/ConciseIPRA/NativeBiInterpretation.lean` and `BiInterpretation.lean` at the pinned commit. Native/iPRA faithful translations and both round trips. The theorem names used in this paper are listed in the provenance section.

[5] ProveIt, `docs/incoming` at the pinned commit: `ipra_computation_normal.zip`, `ipra_counter_evaluation_2026-11-11.zip`, `ipra_reduced_clause_calculus_2026-11-11.zip`, `ipra_rootfree_clausal_2026-11-11.zip`, and `ipra_uniform_step.zip`. Pinned incoming archive directory. The reported complete-program metrics and source hashes are preserved in `evidence/incoming_metrics.json`.

[6] ProveIt, *Computation-normal iPRA and a smaller validated checker*, the first archive in reference [5]. Its fresh-zero argument is a related use of aliases; the present principal calculus does not use computation normalization.

A The complete trusted checker

The listing below is the entire standalone acceptance program. The proof producer, translations, and test utilities are not part of its trusted code. This is the readable source; measurements in the paper apply the stated normalization to the whole module.

```
"""Raw clausal iPRA with computation and root/parameter equality.

Terms: 0, negative free levels, positive bound indices, [0,t] (S),
[n,a,h] (I). Formulas: [s,t] (equality), [B] (D).
Contexts contain reusable (antecedent, one-variable body) clauses.
Input is exactly {'goal': closed_formula, 'proof': instruction_tree}.
Instructions: c(counter,seed,body), x(major,minor),
u(clause_index,target,premise), e(goal,major,branch), i(goal,witness,branch).
All terms are raw. There is no computation normalization or hidden oracle.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def successor(a):
    return [0, a]


def valid(a, scope, depth=0, formula=False):
    if type(a) is int:
        return not formula and -scope <= a <= depth
    return (type(a) is list
            and len(a) in ((1, 2) if formula else (2, 3))
            and (formula or len(a) == 3 or a[0] == 0)
            and all(valid(t, scope,
                          depth + 2 * (len(a) == 1) + (len(a) == 3 and i == 2),
                          len(a) == 1)
                    for i, t in enumerate(a)))


def opening(a, value, index=1):
    if type(a) is int:
        return value if a == index else a - (index > 0 and a > index)
    size = len(a)
    return [opening(t, value,
                    index + (index > 0) * (2 * (size == 1) + (size == 3 and i == 2)))
            for i, t in enumerate(a)]


def derive(proof, context, scope):
    tag, *args = proof
    if tag == 'u':
        index, witness, premise = args
        if (type(index) is not int or not 0 <= index < len(context)
                or not valid(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'c':
        counter, seed, body = args
        term = [counter, seed, body]
        if not valid(term, scope):
            fail()
        if counter == 0:
```

```
            return [term, seed]
        _, counter = counter
        return [term, opening(body, [counter, seed, body])]
    if tag == 'x':
        major, minor = args
        s, t = derive(major, context, scope)
        u, r = derive(minor, context, scope)
        return [r, t if u == s else opening(u, t, s)
                if type(s) is int and s < 0 else u]
    if tag in ('e', 'i'):
        goal, data, branch = args
        if not valid(goal, scope, formula=True):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 2)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not valid(data, scope):
                fail()
            body = opening(goal[0], data, 2)
            eta = fresh if len(goal[0]) == 2 and data == 0 else 0
            clause = ([0, 0], opening(body, eta))
            target = opening(body, successor(fresh))
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if set(data) != {'goal', 'proof'}:
        fail()
    if not valid(data['goal'], 0, formula=True):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```